Resolutions

Release 10.10

The Sage Development Team

Oct 28, 2026

CONTENTS

1 Free resolutions 3

2 Graded free resolutions 13

3 Indices and Tables 21

Python Module Index 23

Index 25

Free and graded resolutions are tools for commutative algebra and algebraic geometry.

CHAPTER

ONE

FREE RESOLUTIONS

Let R be a commutative ring. A finite free resolution of an R-module M is a chain complex of free R-modules

$$0 \to R^{n_k} \xrightarrow{d_k} \cdots \xrightarrow{d_2} R^{n_1} \xrightarrow{d_1} R^{n_0}$$

terminating with a zero module at the end that is exact (all homology groups are zero) such that the image of d_1 is M.

EXAMPLES:

```
sage: from sage.homology.free_resolution import FreeResolution
sage: S.<x,y,z,w> = PolynomialRing(QQ)
sage: m = matrix(S, 1, [z^2 - y*w, y*z - x*w, y^2 - x*z]).transpose()
sage: r = FreeResolution(m, name='S'); r
S^1 <-- S^3 <-- S^2 <-- 0

sage: I = S.ideal([y*w - z^2, -x*w + y*z, x*z - y^2])
sage: r = I.free_resolution(); r
S^1 <-- S^3 <-- S^2 <-- 0
```

```
>>> from sage.all import *
>>> from sage.homology.free_resolution import FreeResolution
>>> S = PolynomialRing(QQ, names=('x', 'y', 'z', 'w',)); (x, y, z, w,) = S._first_
↪ngens(4)
>>> m = matrix(S, Integer(1), [z**Integer(2) - y*w, y*z - x*w, y**Integer(2) - x*z]).
↪transpose()
>>> r = FreeResolution(m, name='S'); r
S^1 <-- S^3 <-- S^2 <-- 0

>>> I = S.ideal([y*w - z**Integer(2), -x*w + y*z, x*z - y**Integer(2)])
>>> r = I.free_resolution(); r
S^1 <-- S^3 <-- S^2 <-- 0
```

```
sage: S.<x,y,z,w> = PolynomialRing(QQ)
sage: I = S.ideal([y*w - z^2, -x*w + y*z, x*z - y^2])
sage: r = I.graded_free_resolution(); r
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
```

```
>>> from sage.all import *
>>> S = PolynomialRing(QQ, names=('x', 'y', 'z', 'w',)); (x, y, z, w,) = S._first_
↪ngens(4)
>>> I = S.ideal([y*w - z**Integer(2), -x*w + y*z, x*z - y**Integer(2)])
```

(continues on next page)

(continued from previous page)

```
>>> r = I.graded_free_resolution(); r
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
```

An example of a minimal free resolution from [CLO2005]:

```
sage: R.<x,y,z,w> = QQ[]
sage: I = R.ideal([y*z - x*w, y^3 - x^2*z, x*z^2 - y^2*w, z^3 - y*w^2])
sage: r = I.free_resolution();  r
S^1 <-- S^4 <-- S^4 <-- S^1 <-- 0
sage: len(r)
3
sage: r.matrix(2)
[-z^2 -x*z  y*w -y^2]
[   y    0   -x    0]
[  -w    y    z    x]
[   0    w    0    z]
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z, w']; (x, y, z, w,) = R._first_ngens(4)
>>> I = R.ideal([y*z - x*w, y**Integer(3) - x**Integer(2)*z, x*z**Integer(2) -␣
↪y**Integer(2)*w, z**Integer(3) - y*w**Integer(2)])
>>> r = I.free_resolution();  r
S^1 <-- S^4 <-- S^4 <-- S^1 <-- 0
>>> len(r)
3
>>> r.matrix(Integer(2))
[-z^2 -x*z  y*w -y^2]
[   y    0   -x    0]
[  -w    y    z    x]
[   0    w    0    z]
```

AUTHORS:

- Kwankyu Lee (2022-05-13): initial version
- Travis Scrimshaw (2022-08-23): refactored for free module inputs

class sage.homology.free_resolution.**FiniteFreeResolution**(*module*, *name='S'*, ***kwds*)

Bases: *FreeResolution*

Finite free resolutions.

The matrix at index i in the list defines the differential map from $(i+1)$-th free module to the i-th free module over the base ring by multiplication on the left. The number of matrices in the list is the length of the resolution. The number of rows and columns of the matrices define the ranks of the free modules in the resolution.

Note that the first matrix in the list defines the differential map at homological index 1.

A subclass must provide a `_maps` attribute that contains a list of the maps defining the resolution.

A subclass can define `_initial_differential` attribute that contains the 0-th differential map whose codomain is the target of the free resolution.

EXAMPLES:

```
sage: from sage.homology.free_resolution import FreeResolution
sage: S.<x,y,z,w> = PolynomialRing(QQ)
sage: I = S.ideal([y*w - z^2, -x*w + y*z, x*z - y^2])
sage: r = FreeResolution(I)
sage: r.differential(0)
Coercion map:
  From: Ambient free module of rank 1 over the integral domain
        Multivariate Polynomial Ring in x, y, z, w over Rational Field
  To:   Quotient module by
        Submodule of Ambient free module of rank 1 over the integral domain
        Multivariate Polynomial Ring in x, y, z, w over Rational Field
        Generated by the rows of the matrix:
        [-z^2 + y*w]
        [ y*z - x*w]
        [-y^2 + x*z]
```

```
>>> from sage.all import *
>>> from sage.homology.free_resolution import FreeResolution
>>> S = PolynomialRing(QQ, names=('x', 'y', 'z', 'w',)); (x, y, z, w,) = S._first_
↪ngens(4)
>>> I = S.ideal([y*w - z**Integer(2), -x*w + y*z, x*z - y**Integer(2)])
>>> r = FreeResolution(I)
>>> r.differential(Integer(0))
Coercion map:
  From: Ambient free module of rank 1 over the integral domain
        Multivariate Polynomial Ring in x, y, z, w over Rational Field
  To:   Quotient module by
        Submodule of Ambient free module of rank 1 over the integral domain
        Multivariate Polynomial Ring in x, y, z, w over Rational Field
        Generated by the rows of the matrix:
        [-z^2 + y*w]
        [ y*z - x*w]
        [-y^2 + x*z]
```

chain_complex()

Return this resolution as a chain complex.

A chain complex in Sage has its own useful methods.

EXAMPLES:

```
sage: S.<x,y,z,w> = PolynomialRing(QQ)
sage: I = S.ideal([y*w - z^2, -x*w + y*z, x*z - y^2])
sage: r = I.graded_free_resolution()
sage: unicode_art(r.chain_complex())
                                                 ⎛-y  x⎞
                                                 ⎜ z -y⎟
           (z^2 - y*w y*z - x*w y^2 - x*z)       ⎝-w  z⎠
 0 <-- C_0 <------------------------------- C_1 <------- C_2 <-- 0
```

```
>>> from sage.all import *
>>> S = PolynomialRing(QQ, names=('x', 'y', 'z', 'w',)); (x, y, z, w,) = S._
↪first_ngens(4)
```

(continues on next page)

(continued from previous page)

```
>>> I = S.ideal([y*w - z**Integer(2), -x*w + y*z, x*z - y**Integer(2)])
>>> r = I.graded_free_resolution()
>>> unicode_art(r.chain_complex())
                                                 ⎛-y  x⎞
                                                 ⎜ z -y⎟
           (z^2 - y*w y*z - x*w y^2 - x*z)       ⎝-w  z⎠
 0 <── C_0 <──────────────────────────────── C_1 <──────── C_2 <── 0
```

differential(*i*)

Return the i-th differential map.

INPUT:

- `i` – positive integer

EXAMPLES:

```
sage: S.<x,y,z,w> = PolynomialRing(QQ)
sage: I = S.ideal([y*w - z^2, -x*w + y*z, x*z - y^2])
sage: r = I.graded_free_resolution()
sage: r
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
sage: r.differential(3)
Free module morphism defined as left-multiplication by the matrix
 []
 Domain:   Ambient free module of rank 0 over the integral domain
           Multivariate Polynomial Ring in x, y, z, w over Rational Field
 Codomain: Ambient free module of rank 2 over the integral domain
           Multivariate Polynomial Ring in x, y, z, w over Rational Field
sage: r.differential(2)
Free module morphism defined as left-multiplication by the matrix
  [-y  x]
  [ z -y]
  [-w  z]
  Domain:   Ambient free module of rank 2 over the integral domain
            Multivariate Polynomial Ring in x, y, z, w over Rational Field
  Codomain: Ambient free module of rank 3 over the integral domain
            Multivariate Polynomial Ring in x, y, z, w over Rational Field
sage: r.differential(1)
Free module morphism defined as left-multiplication by the matrix
  [z^2 - y*w y*z - x*w y^2 - x*z]
  Domain:   Ambient free module of rank 3 over the integral domain
            Multivariate Polynomial Ring in x, y, z, w over Rational Field
  Codomain: Ambient free module of rank 1 over the integral domain
            Multivariate Polynomial Ring in x, y, z, w over Rational Field
sage: r.differential(0)
Coercion map:
  From: Ambient free module of rank 1 over the integral domain
        Multivariate Polynomial Ring in x, y, z, w over Rational Field
  To:   Quotient module by
        Submodule of Ambient free module of rank 1 over the integral domain
        Multivariate Polynomial Ring in x, y, z, w over Rational Field
        Generated by the rows of the matrix:
```

(continues on next page)

(continued from previous page)

```
        [-z^2 + y*w]
        [ y*z - x*w]
        [-y^2 + x*z]
```

```
>>> from sage.all import *
>>> S = PolynomialRing(QQ, names=('x', 'y', 'z', 'w',)); (x, y, z, w,) = S._
↪first_ngens(4)
>>> I = S.ideal([y*w - z**Integer(2), -x*w + y*z, x*z - y**Integer(2)])
>>> r = I.graded_free_resolution()
>>> r
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
>>> r.differential(Integer(3))
Free module morphism defined as left-multiplication by the matrix
 []
 Domain:   Ambient free module of rank 0 over the integral domain
           Multivariate Polynomial Ring in x, y, z, w over Rational Field
 Codomain: Ambient free module of rank 2 over the integral domain
           Multivariate Polynomial Ring in x, y, z, w over Rational Field
>>> r.differential(Integer(2))
Free module morphism defined as left-multiplication by the matrix
  [-y  x]
  [ z -y]
  [-w  z]
  Domain:   Ambient free module of rank 2 over the integral domain
            Multivariate Polynomial Ring in x, y, z, w over Rational Field
  Codomain: Ambient free module of rank 3 over the integral domain
            Multivariate Polynomial Ring in x, y, z, w over Rational Field
>>> r.differential(Integer(1))
Free module morphism defined as left-multiplication by the matrix
  [z^2 - y*w y*z - x*w y^2 - x*z]
  Domain:   Ambient free module of rank 3 over the integral domain
            Multivariate Polynomial Ring in x, y, z, w over Rational Field
  Codomain: Ambient free module of rank 1 over the integral domain
            Multivariate Polynomial Ring in x, y, z, w over Rational Field
>>> r.differential(Integer(0))
Coercion map:
  From: Ambient free module of rank 1 over the integral domain
        Multivariate Polynomial Ring in x, y, z, w over Rational Field
  To:   Quotient module by
        Submodule of Ambient free module of rank 1 over the integral domain
        Multivariate Polynomial Ring in x, y, z, w over Rational Field
        Generated by the rows of the matrix:
        [-z^2 + y*w]
        [ y*z - x*w]
        [-y^2 + x*z]
```

matrix(i)

Return the matrix representing the i-th differential map.

INPUT:

- `i` – positive integer

EXAMPLES:

```
sage: S.<x,y,z,w> = PolynomialRing(QQ)
sage: I = S.ideal([y*w - z^2, -x*w + y*z, x*z - y^2])
sage: r = I.graded_free_resolution(); r
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
sage: r.matrix(3)
[]
sage: r.matrix(2)
[-y  x]
[ z -y]
[-w  z]
sage: r.matrix(1)
[z^2 - y*w y*z - x*w y^2 - x*z]
```

```
>>> from sage.all import *
>>> S = PolynomialRing(QQ, names=('x', 'y', 'z', 'w',)); (x, y, z, w,) = S._
↪first_ngens(4)
>>> I = S.ideal([y*w - z**Integer(2), -x*w + y*z, x*z - y**Integer(2)])
>>> r = I.graded_free_resolution(); r
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
>>> r.matrix(Integer(3))
[]
>>> r.matrix(Integer(2))
[-y  x]
[ z -y]
[-w  z]
>>> r.matrix(Integer(1))
[z^2 - y*w y*z - x*w y^2 - x*z]
```

class sage.homology.free_resolution.**FiniteFreeResolution_free_module**(*module*, *name='S'*, ***kwds*)

Bases: *FiniteFreeResolution*

Free resolutions of a free module.

INPUT:

- `module` – a free module or ideal over a PID
- `name` – the name of the base ring

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: M = R^3
sage: v = M([x^2, 2*x^2, 3*x^2])
sage: w = M([0, x, 2*x])
sage: S = M.submodule([v, w]); S
Free module of degree 3 and rank 2 over
 Univariate Polynomial Ring in x over Rational Field
 Echelon basis matrix:
 [  x^2 2*x^2 3*x^2]
 [    0     x   2*x]
sage: res = S.free_resolution(); res
S^3 <-- S^2 <-- 0
sage: ascii_art(res.chain_complex())
```

(continues on next page)

(continued from previous page)

```
            [  x^2     0]
            [2*x^2     x]
            [3*x^2   2*x]
 0 <-- C_0 <-------------- C_1 <-- 0

sage: R.<x> = PolynomialRing(QQ)
sage: I = R.ideal([x^4 + 3*x^2 + 2])
sage: res = I.free_resolution(); res
S^1 <-- S^1 <-- 0
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> M = R**Integer(3)
>>> v = M([x**Integer(2), Integer(2)*x**Integer(2), Integer(3)*x**Integer(2)])
>>> w = M([Integer(0), x, Integer(2)*x])
>>> S = M.submodule([v, w]); S
Free module of degree 3 and rank 2 over
 Univariate Polynomial Ring in x over Rational Field
 Echelon basis matrix:
 [  x^2 2*x^2 3*x^2]
 [    0     x   2*x]
>>> res = S.free_resolution(); res
S^3 <-- S^2 <-- 0
>>> ascii_art(res.chain_complex())
            [  x^2     0]
            [2*x^2     x]
            [3*x^2   2*x]
 0 <-- C_0 <-------------- C_1 <-- 0

>>> R = PolynomialRing(QQ, names=('x',)); (x,) = R._first_ngens(1)
>>> I = R.ideal([x**Integer(4) + Integer(3)*x**Integer(2) + Integer(2)])
>>> res = I.free_resolution(); res
S^1 <-- S^1 <-- 0
```

class sage.homology.free_resolution.**FiniteFreeResolution_singular**(*module*, *name='S'*, *algorithm='heuristic'*, ***kwds*)

Bases: *FiniteFreeResolution*

Minimal free resolutions of ideals or submodules of free modules of multivariate polynomial rings implemented in Singular.

INPUT:

- `module` – a submodule of a free module M of rank n over S or an ideal of a multi-variate polynomial ring
- `name` – string (optional); name of the base ring
- `algorithm` – (default: `'heuristic'`) Singular algorithm to compute a resolution of `ideal`

OUTPUT: a minimal free resolution of the ideal

If `module` is an ideal of S, it is considered as a submodule of a free module of rank 1 over S.

The available algorithms and the corresponding Singular commands are shown below:

algorithm	Singular commands
minimal	mres(ideal)
shreyer	minres(sres(std(ideal)))
standard	minres(nres(std(ideal)))
heuristic	minres(res(std(ideal)))

EXAMPLES:

```
sage: from sage.homology.free_resolution import FreeResolution
sage: S.<x,y,z,w> = PolynomialRing(QQ)
sage: I = S.ideal([y*w - z^2, -x*w + y*z, x*z - y^2])
sage: r = FreeResolution(I); r
S^1 <-- S^3 <-- S^2 <-- 0
sage: len(r)
2
```

```
>>> from sage.all import *
>>> from sage.homology.free_resolution import FreeResolution
>>> S = PolynomialRing(QQ, names=('x', 'y', 'z', 'w',)); (x, y, z, w,) = S._first_
↪ngens(4)
>>> I = S.ideal([y*w - z**Integer(2), -x*w + y*z, x*z - y**Integer(2)])
>>> r = FreeResolution(I); r
S^1 <-- S^3 <-- S^2 <-- 0
>>> len(r)
2
```

```
sage: FreeResolution(I, algorithm='minimal')
S^1 <-- S^3 <-- S^2 <-- 0
sage: FreeResolution(I, algorithm='shreyer')
S^1 <-- S^3 <-- S^2 <-- 0
sage: FreeResolution(I, algorithm='standard')
S^1 <-- S^3 <-- S^2 <-- 0
sage: FreeResolution(I, algorithm='heuristic')
S^1 <-- S^3 <-- S^2 <-- 0
```

```
>>> from sage.all import *
>>> FreeResolution(I, algorithm='minimal')
S^1 <-- S^3 <-- S^2 <-- 0
>>> FreeResolution(I, algorithm='shreyer')
S^1 <-- S^3 <-- S^2 <-- 0
>>> FreeResolution(I, algorithm='standard')
S^1 <-- S^3 <-- S^2 <-- 0
>>> FreeResolution(I, algorithm='heuristic')
S^1 <-- S^3 <-- S^2 <-- 0
```

We can also construct a resolution by passing in a matrix defining the initial differential:

```
sage: m = matrix(S, 1, [z^2 - y*w, y*z - x*w, y^2 - x*z]).transpose()
sage: r = FreeResolution(m, name='S'); r
S^1 <-- S^3 <-- S^2 <-- 0
sage: r.matrix(1)
```

(continues on next page)

(continued from previous page)

```
[z^2 - y*w y*z - x*w y^2 - x*z]
```

```
>>> from sage.all import *
>>> m = matrix(S, Integer(1), [z**Integer(2) - y*w, y*z - x*w, y**Integer(2) -␣
↪x*z]).transpose()
>>> r = FreeResolution(m, name='S'); r
S^1 <-- S^3 <-- S^2 <-- 0
>>> r.matrix(Integer(1))
[z^2 - y*w y*z - x*w y^2 - x*z]
```

An additional construction is using a submodule of a free module:

```
sage: M = m.image()
sage: r = FreeResolution(M, name='S'); r
S^1 <-- S^3 <-- S^2 <-- 0
```

```
>>> from sage.all import *
>>> M = m.image()
>>> r = FreeResolution(M, name='S'); r
S^1 <-- S^3 <-- S^2 <-- 0
```

A nonhomogeneous ideal:

```
sage: I = S.ideal([z^2 - y*w, y*z - x*w, y^2 - x])
sage: R = FreeResolution(I); R
S^1 <-- S^3 <-- S^3 <-- S^1 <-- 0
sage: R.matrix(2)
[ y*z - x*w    y^2 - x          0]
[-z^2 + y*w          0    y^2 - x]
[         0 -z^2 + y*w -y*z + x*w]
sage: R.matrix(3)
[   y^2 - x]
[-y*z + x*w]
[ z^2 - y*w]
```

```
>>> from sage.all import *
>>> I = S.ideal([z**Integer(2) - y*w, y*z - x*w, y**Integer(2) - x])
>>> R = FreeResolution(I); R
S^1 <-- S^3 <-- S^3 <-- S^1 <-- 0
>>> R.matrix(Integer(2))
[ y*z - x*w    y^2 - x          0]
[-z^2 + y*w          0    y^2 - x]
[         0 -z^2 + y*w -y*z + x*w]
>>> R.matrix(Integer(3))
[   y^2 - x]
[-y*z + x*w]
[ z^2 - y*w]
```

class sage.homology.free_resolution.**FreeResolution**(*module*, *name='S'*, ***kwds*)

Bases: SageObject

A free resolution.

Let R be a commutative ring. A *free resolution* of an R-module M is a (possibly infinite) chain complex of free R-modules

$$\cdots \to R^{n_k} \xrightarrow{d_k} \cdots \xrightarrow{d_2} R^{n_1} \xrightarrow{d_1} R^{n_0}$$

that is exact (all homology groups are zero) such that the image of d_1 is M.

differential(i)

Return the i-th differential map.

INPUT:

- `i` – positive integer

target()

Return the codomain of the 0-th differential map.

The codomain of the 0-th differential map is the cokernel of the first differential map.

EXAMPLES:

```
sage: S.<x,y,z,w> = PolynomialRing(QQ)
sage: I = S.ideal([y*w - z^2, -x*w + y*z, x*z - y^2])
sage: r = I.graded_free_resolution()
sage: r
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
sage: r.target()
Quotient module by
 Submodule of Ambient free module of rank 1 over the integral domain
  Multivariate Polynomial Ring in x, y, z, w over Rational Field
  Generated by the rows of the matrix:
  [-z^2 + y*w]
  [ y*z - x*w]
  [-y^2 + x*z]
```

```
>>> from sage.all import *
>>> S = PolynomialRing(QQ, names=('x', 'y', 'z', 'w',)); (x, y, z, w,) = S._
↪first_ngens(4)
>>> I = S.ideal([y*w - z**Integer(2), -x*w + y*z, x*z - y**Integer(2)])
>>> r = I.graded_free_resolution()
>>> r
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
>>> r.target()
Quotient module by
 Submodule of Ambient free module of rank 1 over the integral domain
  Multivariate Polynomial Ring in x, y, z, w over Rational Field
  Generated by the rows of the matrix:
  [-z^2 + y*w]
  [ y*z - x*w]
  [-y^2 + x*z]
```

CHAPTER

TWO

GRADED FREE RESOLUTIONS

Let R be a commutative ring. A graded free resolution of a graded R-module M is a *free resolution* such that all maps are homogeneous module homomorphisms.

EXAMPLES:

```
sage: S.<x,y,z,w> = PolynomialRing(QQ)
sage: I = S.ideal([y*w - z^2, -x*w + y*z, x*z - y^2])
sage: r = I.graded_free_resolution(algorithm='minimal')
sage: r
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
sage: I.graded_free_resolution(algorithm='shreyer')
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
sage: I.graded_free_resolution(algorithm='standard')
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
sage: I.graded_free_resolution(algorithm='heuristic')
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
```

```
>>> from sage.all import *
>>> S = PolynomialRing(QQ, names=('x', 'y', 'z', 'w',)); (x, y, z, w,) = S._first_
↪ngens(4)
>>> I = S.ideal([y*w - z**Integer(2), -x*w + y*z, x*z - y**Integer(2)])
>>> r = I.graded_free_resolution(algorithm='minimal')
>>> r
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
>>> I.graded_free_resolution(algorithm='shreyer')
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
>>> I.graded_free_resolution(algorithm='standard')
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
>>> I.graded_free_resolution(algorithm='heuristic')
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
```

```
sage: d = r.differential(2)
sage: d
Free module morphism defined as left-multiplication by the matrix
  [ y  x]
  [-z -y]
  [ w  z]
  Domain:   Ambient free module of rank 2 over the integral domain
            Multivariate Polynomial Ring in x, y, z, w over Rational Field
  Codomain: Ambient free module of rank 3 over the integral domain
```

(continues on next page)

(continued from previous page)

```
            Multivariate Polynomial Ring in x, y, z, w over Rational Field
sage: d.image()
Submodule of Ambient free module of rank 3 over the integral domain
 Multivariate Polynomial Ring in x, y, z, w over Rational Field
Generated by the rows of the matrix:
[ y -z  w]
[ x -y  z]
sage: m = d.image()
sage: m.graded_free_resolution(shifts=(2,2,2))
S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
```

```
>>> from sage.all import *
>>> d = r.differential(Integer(2))
>>> d
Free module morphism defined as left-multiplication by the matrix
  [ y  x]
  [-z -y]
  [ w  z]
  Domain:   Ambient free module of rank 2 over the integral domain
            Multivariate Polynomial Ring in x, y, z, w over Rational Field
  Codomain: Ambient free module of rank 3 over the integral domain
            Multivariate Polynomial Ring in x, y, z, w over Rational Field
>>> d.image()
Submodule of Ambient free module of rank 3 over the integral domain
 Multivariate Polynomial Ring in x, y, z, w over Rational Field
Generated by the rows of the matrix:
[ y -z  w]
[ x -y  z]
>>> m = d.image()
>>> m.graded_free_resolution(shifts=(Integer(2),Integer(2),Integer(2)))
S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
```

An example of multigraded resolution from Example 9.1 of [MilStu2005]:

```
sage: R.<s,t> = QQ[]
sage: S.<a,b,c,d> = QQ[]
sage: phi = S.hom([s, s*t, s*t^2, s*t^3])
sage: I = phi.kernel(); I                                                    #␣
↪needs sage.rings.function_field
Ideal (c^2 - b*d, b*c - a*d, b^2 - a*c) of
 Multivariate Polynomial Ring in a, b, c, d over Rational Field
sage: P3 = ProjectiveSpace(S)
sage: C = P3.subscheme(I)  # twisted cubic curve
sage: r = I.graded_free_resolution(degrees=[(1,0), (1,1), (1,2), (1,3)])
sage: r
S((0, 0)) <-- S((-2, -4))⊕S((-2, -3))⊕S((-2, -2)) <-- S((-3, -5))⊕S((-3, -4)) <-- 0
sage: r.K_polynomial(names='s,t')
s^3*t^5 + s^3*t^4 - s^2*t^4 - s^2*t^3 - s^2*t^2 + 1
```

```
>>> from sage.all import *
>>> R = QQ['s, t']; (s, t,) = R._first_ngens(2)
>>> S = QQ['a, b, c, d']; (a, b, c, d,) = S._first_ngens(4)
```

(continues on next page)

(continued from previous page)

```
>>> phi = S.hom([s, s*t, s*t**Integer(2), s*t**Integer(3)])
>>> I = phi.kernel(); I                                                    #␣
↪needs sage.rings.function_field
Ideal (c^2 - b*d, b*c - a*d, b^2 - a*c) of
 Multivariate Polynomial Ring in a, b, c, d over Rational Field
>>> P3 = ProjectiveSpace(S)
>>> C = P3.subscheme(I)  # twisted cubic curve
>>> r = I.graded_free_resolution(degrees=[(Integer(1),Integer(0)), (Integer(1),
↪Integer(1)), (Integer(1),Integer(2)), (Integer(1),Integer(3))])
>>> r
S((0, 0)) <-- S((-2, -4))⊕S((-2, -3))⊕S((-2, -2)) <-- S((-3, -5))⊕S((-3, -4)) <-- 0
>>> r.K_polynomial(names='s,t')
s^3*t^5 + s^3*t^4 - s^2*t^4 - s^2*t^3 - s^2*t^2 + 1
```

AUTHORS:

- Kwankyu Lee (2022-05): initial version
- Travis Scrimshaw (2022-08-23): refactored for free module inputs

class sage.homology.graded_resolution.**GradedFiniteFreeResolution**(*module*, *degrees=None*, *shifts=None*, *name='S'*, ***kwds*)

Bases: *FiniteFreeResolution*

Graded finite free resolutions.

INPUT:

- `module` – a homogeneous submodule of a free module M of rank n over S or a homogeneous ideal of a multivariate polynomial ring S
- `degrees` – (default: a list with all entries 1) a list of integers or integer vectors giving degrees of variables of S
- `shifts` – list of integers or integer vectors giving shifts of degrees of n summands of the free module M; this is a list of zero degrees of length n by default
- `name` – string; name of the base ring

K_polynomial(*names=None*)

Return the K-polynomial of this resolution.

INPUT:

- `names` – (optional) a string of names of the variables of the K-polynomial

EXAMPLES:

```
sage: S.<x,y,z,w> = PolynomialRing(QQ)
sage: I = S.ideal([y*w - z^2, -x*w + y*z, x*z - y^2])
sage: r = I.graded_free_resolution()
sage: r.K_polynomial()
2*t^3 - 3*t^2 + 1
```

```
>>> from sage.all import *
>>> S = PolynomialRing(QQ, names=('x', 'y', 'z', 'w',)); (x, y, z, w,) = S._
↪first_ngens(4)
```

(continues on next page)

(continued from previous page)

```
>>> I = S.ideal([y*w - z**Integer(2), -x*w + y*z, x*z - y**Integer(2)])
>>> r = I.graded_free_resolution()
>>> r.K_polynomial()
2*t^3 - 3*t^2 + 1
```

betti(*i*, *a=None*)

Return the i-th Betti number in degree a.

INPUT:

- `i` – nonnegative integer
- `a` – a degree; if `None`, return Betti numbers in all degrees

EXAMPLES:

```
sage: S.<x,y,z,w> = PolynomialRing(QQ)
sage: I = S.ideal([y*w - z^2, -x*w + y*z, x*z - y^2])
sage: r = I.graded_free_resolution()
sage: r.betti(0)
{0: 1}
sage: r.betti(1)
{2: 3}
sage: r.betti(2)
{3: 2}
sage: r.betti(1, 0)
0
sage: r.betti(1, 1)
0
sage: r.betti(1, 2)
3
```

```
>>> from sage.all import *
>>> S = PolynomialRing(QQ, names=('x', 'y', 'z', 'w',)); (x, y, z, w,) = S._
↪first_ngens(4)
>>> I = S.ideal([y*w - z**Integer(2), -x*w + y*z, x*z - y**Integer(2)])
>>> r = I.graded_free_resolution()
>>> r.betti(Integer(0))
{0: 1}
>>> r.betti(Integer(1))
{2: 3}
>>> r.betti(Integer(2))
{3: 2}
>>> r.betti(Integer(1), Integer(0))
0
>>> r.betti(Integer(1), Integer(1))
0
>>> r.betti(Integer(1), Integer(2))
3
```

shifts(*i*)

Return the shifts of `self`.

EXAMPLES:

```
sage: S.<x,y,z,w> = PolynomialRing(QQ)
sage: I = S.ideal([y*w - z^2, -x*w + y*z, x*z - y^2])
sage: r = I.graded_free_resolution()
sage: r.shifts(0)
[0]
sage: r.shifts(1)
[2, 2, 2]
sage: r.shifts(2)
[3, 3]
sage: r.shifts(3)
[]
```

```
>>> from sage.all import *
>>> S = PolynomialRing(QQ, names=('x', 'y', 'z', 'w',)); (x, y, z, w,) = S._
↪first_ngens(4)
>>> I = S.ideal([y*w - z**Integer(2), -x*w + y*z, x*z - y**Integer(2)])
>>> r = I.graded_free_resolution()
>>> r.shifts(Integer(0))
[0]
>>> r.shifts(Integer(1))
[2, 2, 2]
>>> r.shifts(Integer(2))
[3, 3]
>>> r.shifts(Integer(3))
[]
```

class sage.homology.graded_resolution.**GradedFiniteFreeResolution_free_module**(*module*, *degrees=None*, **args*, ***kwds*)

Bases: *GradedFiniteFreeResolution*, *FiniteFreeResolution_free_module*

Graded free resolution of free modules.

EXAMPLES:

```
sage: from sage.homology.free_resolution import FreeResolution
sage: R.<x> = QQ[]
sage: M = matrix([[x^3, 3*x^3, 5*x^3],
....:             [0, x, 2*x]])
sage: res = FreeResolution(M, graded=True); res
S(0)⊕S(0)⊕S(0) <-- S(-3)⊕S(-1) <-- 0
```

```
>>> from sage.all import *
>>> from sage.homology.free_resolution import FreeResolution
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> M = matrix([[x**Integer(3), Integer(3)*x**Integer(3),␣
↪Integer(5)*x**Integer(3)],
...             [Integer(0), x, Integer(2)*x]])
>>> res = FreeResolution(M, graded=True); res
S(0)⊕S(0)⊕S(0) <-- S(-3)⊕S(-1) <-- 0
```

class sage.homology.graded_resolution.**GradedFiniteFreeResolution_singular**(*module*, *degrees=None*, *shifts=None*, *name='S'*, *algorithm='heuristic'*, ***kwds*)

Bases: *GradedFiniteFreeResolution*, *FiniteFreeResolution_singular*

Graded free resolutions of submodules and ideals of multivariate polynomial rings implemented using Singular.

INPUT:

- `module` – a homogeneous submodule of a free module M of rank n over S or a homogeneous ideal of a multivariate polynomial ring S
- `degrees` – (default: a list with all entries 1) a list of integers or integer vectors giving degrees of variables of S
- `shifts` – list of integers or integer vectors giving shifts of degrees of n summands of the free module M; this is a list of zero degrees of length n by default
- `name` – string; name of the base ring
- `algorithm` – Singular algorithm to compute a resolution of `ideal`

OUTPUT: a graded minimal free resolution of `ideal`

If `module` is an ideal of S, it is considered as a submodule of a free module of rank 1 over S.

The degrees given to the variables of S are integers or integer vectors of the same length. In the latter case, S is said to be multigraded, and the resolution is a multigraded free resolution. The standard grading where all variables have degree 1 is used if the degrees are not specified.

A summand of the graded free module M is a shifted (or twisted) module of rank one over S, denoted $S(-d)$ with shift d.

The computation of the resolution is done by using `libSingular`. Different Singular algorithms can be chosen for best performance. The available algorithms and the corresponding Singular commands are shown below:

algorithm	Singular commands
`minimal`	`mres(ideal)`
`shreyer`	`minres(sres(std(ideal)))`
`standard`	`minres(nres(std(ideal)))`
`heuristic`	`minres(res(std(ideal)))`

EXAMPLES:

```
sage: S.<x,y,z,w> = PolynomialRing(QQ)
sage: I = S.ideal([y*w - z^2, -x*w + y*z, x*z - y^2])
sage: r = I.graded_free_resolution(); r
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
sage: len(r)
2

sage: I = S.ideal([z^2 - y*w, y*z - x*w, y - x])
sage: I.is_homogeneous()
True
```

(continues on next page)

(continued from previous page)

```
sage: r = I.graded_free_resolution(); r
S(0) <-- S(-1)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3)⊕S(-4) <-- S(-5) <-- 0
```

```
>>> from sage.all import *
>>> S = PolynomialRing(QQ, names=('x', 'y', 'z', 'w',)); (x, y, z, w,) = S._first_
↪ngens(4)
>>> I = S.ideal([y*w - z**Integer(2), -x*w + y*z, x*z - y**Integer(2)])
>>> r = I.graded_free_resolution(); r
S(0) <-- S(-2)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3) <-- 0
>>> len(r)
2

>>> I = S.ideal([z**Integer(2) - y*w, y*z - x*w, y - x])
>>> I.is_homogeneous()
True
>>> r = I.graded_free_resolution(); r
S(0) <-- S(-1)⊕S(-2)⊕S(-2) <-- S(-3)⊕S(-3)⊕S(-4) <-- S(-5) <-- 0
```

CHAPTER

THREE

INDICES AND TABLES

- Index
- Module Index
- Search Page

PYTHON MODULE INDEX

h

sage.homology.free_resolution, 3
sage.homology.graded_resolution, 13

INDEX

B

betti() (*sage.homology.graded_resolution.GradedFiniteFreeResolution method*), 16

C

chain_complex() (*sage.homology.free_resolution.FiniteFreeResolution method*), 5

D

differential() (*sage.homology.free_resolution.FiniteFreeResolution method*), 6
differential() (*sage.homology.free_resolution.FreeResolution method*), 12

F

FiniteFreeResolution (*class in sage.homology.free_resolution*), 4
FiniteFreeResolution_free_module (*class in sage.homology.free_resolution*), 8
FiniteFreeResolution_singular (*class in sage.homology.free_resolution*), 9
FreeResolution (*class in sage.homology.free_resolution*), 11

G

GradedFiniteFreeResolution (*class in sage.homology.graded_resolution*), 15
GradedFiniteFreeResolution_free_module (*class in sage.homology.graded_resolution*), 17
GradedFiniteFreeResolution_singular (*class in sage.homology.graded_resolution*), 17

K

K_polynomial() (*sage.homology.graded_resolution.GradedFiniteFreeResolution method*), 15

M

matrix() (*sage.homology.free_resolution.FiniteFreeResolution method*), 7
module
    sage.homology.free_resolution, 3
    sage.homology.graded_resolution, 13

S

sage.homology.free_resolution
    module, 3
sage.homology.graded_resolution
    module, 13
shifts() (*sage.homology.graded_resolution.GradedFiniteFreeResolution method*), 16

T

target() (*sage.homology.free_resolution.FreeResolution method*), 12